

2026 Computer Graphics Project 2 Report

Name: MinYong Seo (서만용)

Student ID: 2022056562

A. Video Link: <https://youtu.be/rUdQQNEqQAQ>

B. Scene Viewer

A. 어떻게 사용자가 카메라를 조작하는지 설명하시오.(e.g., mouse, keyboard, or both)
키보드와 마우스 모두 사용하여 조작할 수 있습니다.

기본적으로 마우스만 사용해서 화면을 드래그하여 조작도 가능하고,
특정 키보드 키를 꾹 누른 상태에서 마우스로 드래그한다면 다른 조작이 이루어지고,
이번 Project2 의 구현물의 특성을 고려해서 추가한 기능인 특정 키보드 키를 꾹
누르고 있으면 일어나는 조작도 있습니다.

키보드 키를 눌러서 viewing mode 를 바꾸는 형태도 고려했으나, 기존 Project1 에서의
카메라 조작 구현이 키보드 키를 꾹 누른 상태에서 일어나도록 작성했었기에
일관성을 위해서 이처럼 구현하였습니다.

B. 각 사용자 입력에 따라 카메라가 어떻게 변화하는지 설명하시오.

- 입력: 마우스 드래그
- 동작: 카메라가 마우스 이동의 반대방향으로
cam_center 을 중심으로 하여 원을 그리며 돔 [Oibit]
- 입력: X 키를 누른 채 마우스 드래그
- 동작: 카메라가 마우스 이동의 반대방향으로 평행이동 [Panning]
- 입력: Z 키를 누른 채 마우스 드래그
- 동작: 마우스 위/아래 방향에 대해서 카메라의 cam_r 값이 증가/감소 [Zoom]
- [New!] 입력: F 키 꾹 누르기
- 동작: 카메라의 cam_center 가 태양의 위치와 같아짐 즉, 태양의 움직임을 따라감
 - 천체의 움직임을 구현했는데, 그 중 중심이 되는 태양을 중점으로 하여 관찰 할 수 있도록 입력을 추가하였습니다.
 - 태양의 Children 들은 태양을 중심으로 하여 공전하기 때문에, 화면의 중심이 태양이 되면 이러한 현상을 더 잘 관찰 할 수 있습니다.

C. Mesh Processing Design

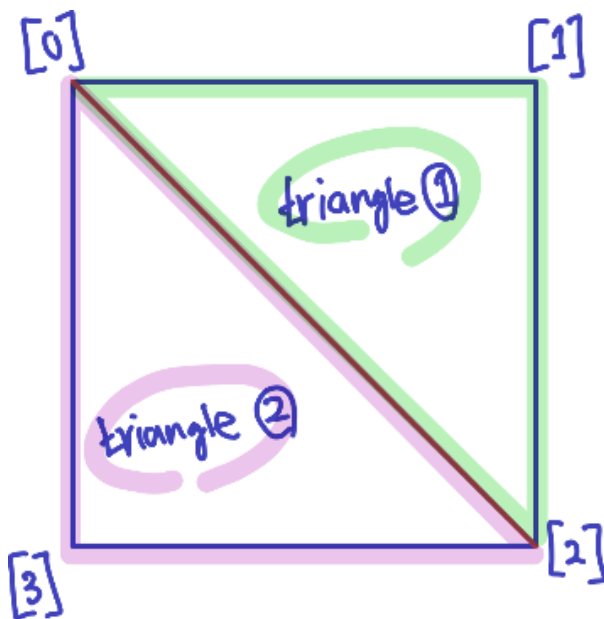
A. face 데이터를 어떻게 처리하는지(triangles, quads, a mix of triangles and quads), 왜 그 방식을 선택했는지 설명하시오.

여러가지 OBJ 파일을 살펴보았을 때, faces 부분에 대한 vertex index 의 개수가 quad 라면 4 개이고, triangle 이라면 3 개임을 알았습니다. 또한 수업 시간에 다루었듯, face index 유형은 (v/vt/vn), (v//vn), (v) 총 3 가지입니다. 하여 이러한 경우들 모두 처리 할 수 있도록 분기 처리를 해주었습니다. 이를 각 line 별로 분기처리를 하기 때문에, face 데이터가 quad/triangle 이건, 데이터 유형이 (v/vt/vn), (v//vn), (v) 이건 각각의 케이스에 대해서 처리가 가능하게 작성하였습니다.

또한 저희 project2 에서는 mtl 파일은 사용하지 않기 때문에, vt 같은 경우는 그냥 parsing 만 되고 버리게 되는 데이터 입니다. 다음은 코드 중 load_obj 의 quad 와 triangle 모두 처리 할 수 있도록 하는 마지막 핵심 2 줄입니다.

```
for i in range(1, len(face_info) - 1):  
    faces.append([face_info[0], face_info[i], face_info[i+1]])
```

0 번 index 와 i, i+1 index 를 하나의 삼각형으로 처리하기 때문에, 그림으로 보면 quad 데이터는 다음과 같은 2 개의 삼각형으로 처리 됩니다.



이러한 방식을 채택한 이유는 매우 간결하고 직관적이기 때문입니다.

또한 수업시간 중 lab check2 에서, 삼각형만 가지고 사각형을 그리라는 요구사항이 있었는데,

그때 당시 위와 같은 로직으로 처리했을 정도로 아주 기본적인 접근이라고 생각합니다.

추가적으로 과제 명세엔 quad 까지만 처리해도 된다고 했지만, 구현 할 때 range 부분을

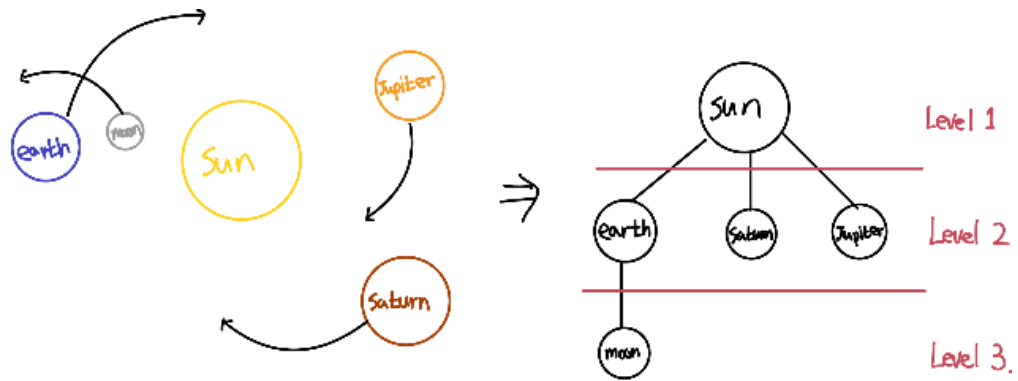
하드코딩하지 않았기 때문에 n-polygon 에 대해서도 제대로 동작하는 구현입니다.

B. 렌더링을 위해 데이터를 어떻게 구성했는지(e.g., vertex duplication, indexing, etc.), 왜 그 방식을 선택했는지 설명하시오.

Vertex duplication 을 선택했습니다. 우선 obj 파일 구조 자체가 한 vertex 에 여러가지 vertex normal 이 대응될 수 있습니다. 즉 기본적으로 obj 파일의 내용을 그대로 받아온다고 하면 vertex duplication 방식이라 할 수 있습니다. 각 vertex 가 어느 face 에 해당되고, 다른 face 데이터들에 대해서 쓰이는지 counting 한 후 모두 averaging 하여 하나의 vertex normal 로 만들어 주어야하는데, 이는 구현적 overhead 라고 생각했습니다. 추가적으로 obj 파일을 선택할 때 low-poly 스타일의 파일들을 선택했기 때문에, rendering 시의 overhead 와 초기 averaging 하는 연산에 대한 overhead 가 크게 다르지 않을 듯 한데, vertex duplication 은 parsing 한 데이터를 곧이 곧대로 쓸 수 있습니다. 무엇보다 기존 project1 코드가 vertex duplication 을 사용하고 있기 때문에, 일관성을 위함도 있습니다.

D. Hierarchical Modeling & Animation Design

A. 모델의 계층 구조를 그림으로 설명하시오.



태양계처럼 태양을 중심으로 각 행성들이 돌고, 그 중 지구에 대해선 달이 지구를 중심으로 회전합니다. 또한 역동성을 추가하기 위해서 태양 역시 world space 의 (0,0,0) 을 기준으로 하여 회전합니다.

B. 변환이 어떻게 적용되는지, 애니메이션이 어떻게 정의되는지 설명하시오.

다음은 transformation 과 animation 이 일어나는 주요 코드 일부분입니다.

```
# animating
t = glfwGetTime()
sun.set_transform(glm.rotate(t * 1, glm.vec3(0, 1, 0)) * glm.translate(glm.vec3(12.0, 0, 0)))
earth.set_transform(glm.rotate(t * 1, glm.vec3(0, 0, 1)) * glm.translate(glm.vec3(0, 6.0, 0)))
saturn.set_transform((glm.rotate(t * 0.5, glm.vec3(0, 0, 1)) * glm.translate(glm.vec3(0, 8.0, 0))))
moon.set_transform((glm.rotate(t * 1.5, glm.vec3(0, 1, 0)) * glm.translate(glm.vec3(3.0, 0, 0))))
jupiter.set_transform((glm.rotate(t * 0.8, glm.vec3(0, 0, 1)) * glm.translate(glm.vec3(0, 10.0, 0))))
sun.update_tree_global_transform()
```

각 node 들은 parent 를 기준으로 하여 일정한 거리를 이동(translate) 하고, 그 후 지정된 축에 대해서 회전(rotation) 합니다. 이는 기본적으로 matrix multiplication 이 뒤에서부터 곱해진다고 생각하는 관점을 채택하는 것입니다. 그 이유는 현재는 reference frame 에 대한 각 object 의 이동이 이루어지고, 이것이 parent 와 child 간 계층적으로 이루어지는 것이지, 새로운 frame 을 정의하는 것이 아니기 때문입니다. 또한 계층적인 구조를 활용하기 때문에, Root 가 되는 sun 에 대한 global transform 만 update 해주어도 recursive 하게 모두 계산됩니다. [각 Node 에 대한 함수는 lab 코드와 같습니다.]

C. 왜 위 방식으로 계층 구조와 애니메이션을 설계했는지 설명하시오.

우선 기본적으로 각 태양을 포함한 각 행성들은 기준에 대한 회전만 합니다. 즉 태양계처럼 설계하였지만, 공전만 하고 자전은 하지 않습니다. 이는 자전까지 시켜버리면 달 같은 경우는 자전에 대한 회전도 반영하여 공전하게 되기 때문에, 이를 적용하여 코드를 실행해보면 한 축에 대해서만 회전하는 것이 아니기 때문에 시각적으로 좋지 않다고 생각했습니다.

E. Lighting Design

A. lighting parameters 를 어떻게 선택했는지 설명하시오.

material_shininess 와 light_color 같은 경우는 기존 lab 코드와 동일하게, 32 로 고정된 값을 사용했습니다. 이유는 바뀌가면서 돌려보았을 때 가장 적절한 값이라고 생각했기 때문입니다.

view_pos, light_pos, light_color material_color 는 모두 uniform variable 로, main 에서 값을 지정할 수 있도록 했습니다. lighting 연산은 모두 Flat+ Phong illumination + Phong Shading 과 동일합니다.

view_pos 는 움직이고 있는 카메라의 eye 와 같도록 매 프레임 마다 업데이트합니다.

Light_pos 는 현재 태양의 위치: sun_pos 와 같도록 매 프레임 마다 업데이트합니다.

제가 구현한 모델은 기본적으로 광원이라고 할 수 있는 물체는 태양뿐이기 때문에 이와 같이 구현했습니다.

light_color 의 초기 값은 (1,1,1)로 백색광인데, 이는 태양이 백색광을 내기 때문입니다. 또한 light_color 값은 매 프레임마다 RGB 각각 점차 낮아지는데, 이는 태양의 힘이 점차 약해지다가 태양계 자체가 죽는 것을 표현하고 싶었기에 이처럼 구현했습니다.

material_color 는 태양을 제외하곤 매 프레임 업데이트 되지는 않고, 각 행성들의 초기 값만 받습니다.

태양에 대한 material_color 인 sun_color 역시 light_color 처럼 값이 점차 낮아지게 되어 태양의 색 역시 검은색으로 바뀌게 됩니다. 이는 shader 설계를 frame 이나 grid 와 같이 빛에 영향을 받지 않는 것 shader, 태양에 대한 shader, 그 외 lighting 연산이 필요한 것들에 대한 shader 로 나누어져있기 때문입니다. 즉, 태양 자체가 광원이기 때문에 태양은 lighting 이 되지 않아서 light_color 의 값을 줄인다고 해서 태양의 색 자체는 변하지 않습니다. 하여 이를 Node class 내 set_color 함수를 추가하여 color_sun 값 자체를 수정할 수 있도록 구현했습니다.

B. 선택한 lighting parameters 가 scene 의 visual appearance 와 readability 에 어떤 영향을 주는지 설명하시오.

A 에서 설명하였듯, 따로 광원이 존재하는 것이 아니라 태양 자체가 광원으로써 역할하기 때문에 각 행성들의 모습이 태양을 바라보는 면만 제대로 보이고, 그 뒷면은 검정색으로 보여 잘 보이지 않습니다. 또한 frame 이나 grid 에 대한 shader 자체를 분리해두었기 때문에 비현실적인 물체들에 대한 작용은 없습니다.

C. 선택한 방식이 scene 의 visibility 와 understanding 을 높이는 이유를 설명하시오.

따로 광원이 존재하는 것이 아니라 태양 자체가 광원으로서 역할하기 때문에 실제로 우주공간에서 태양계를 관찰하는 듯한 효과를 줄 수 있습니다. 만일 orbit camera 에서 FPS camera 로 전환시킬 수 있는 기능을 추가하고, 해당 카메라를 지구의 어떤 한 곳에 고정시켜두었다면 지구 위에서 낮과 밤이 태양과의 상대적인 위치에 따라 어떻게 바뀌는지를 볼 수 있었을 것 입니다. [물론 자전을 구현하지 않아서 올바른 시뮬레이팅은 아닙니다.]

또한 시간이 지날 수록 빛이 약해지는 효과를 추가하여 마치 태양계가 죽어가는 시뮬레이션과 같은 애니메이션을 가집니다.

F. Others (선택)

그래픽스 실력이 부족하여 기존 계획했던 대로 결과물을 구현하지는 못했습니다...

조금 조출하게 구현결과가 나왔는데, 기존에 구현하고자 했던 건 다음 유튜브 링크와 같습니다.

https://youtu.be/0jHsq36_NTU?si=dT_wh0R6KYmdgQsl

태양을 공전하는 행성들의 자취가 나선형으로 나타나게 되는 helical model 을 만들기 위해 기존 hierarchical model 에서 parent 의 이전 위치에 대해서 조금 차이나는 위치를 가지는 child 를 만드는게 쉽지 않았습니다.

과제 제출은 시간이 부족한 관계로 이대로 하지만, 나중에 시간이 난다면 이러한 로직까지 추가적으로 구현해보면 정말 좋을 것 같습니다!